Project Report

Winter Semester 2025-26

Computer Vision (CSE 3010)

SLOT – **D11+D12**

Project Type: Bring Your Own Project (BYOP)

Object Detection Project

Submitted by:

Name: **Aarya Butolia**

Reg. No: **23BAI10414**

Submitted to:

Faculty: Dr. Rajneesh Patel

1. Introduction

Object detection is one of the most important tasks in **Computer Vision**, a field of Artificial Intelligence that enables machines to interpret and understand visual information from the real world. It combines **image classification** and **localization**, meaning it not only identifies what objects are present in an image but also determines **where those objects are located**.

Traditional computer vision techniques relied on handcrafted features such as edges, textures, and shapes. However, these methods were slow and less accurate when dealing with complex scenes. With the advancement of **Deep Learning**, modern object detection systems use **Convolutional Neural Networks (CNNs)** to automatically learn features from images.

This project focuses on implementing a **real-time object detection system using** YOLOv8. The term **YOLO (You Only Look Once)** represents a single-stage detection method that processes the entire image at once, making it extremely fast compared to earlier multi-stage detectors.

The developed system supports:

- Static image detection
- Video-based detection
- Real-time webcam detection

This project demonstrates how modern deep learning models can be used to build efficient and scalable real-time detection pipelines suitable for real-world applications such as:

- Smart surveillance
- Automated monitoring
- Traffic analysis

- Industrial inspection

The main focus is to design a system that is **fast, accurate, flexible, and user-friendly**, making it suitable for deployment in practical environments.

---

2. Problem Statement

Manual object detection from images or videos is inefficient, time-consuming, and prone to human error. In many real-world scenarios such as surveillance, traffic monitoring, and industrial automation, thousands of images or video frames need to be processed continuously. Manual monitoring is not feasible in such situations.

Additionally, with the rapid increase in digital content generation, there is a strong need for **automated intelligent systems** capable of identifying objects in real time.

The key challenges addressed in this project include:

1. Real-Time Detection

Many traditional object detection systems are slow and cannot process live video streams efficiently. This project aims to build a system capable of processing frames quickly while maintaining high detection accuracy.

2. Multi-Input Processing

Most existing systems handle only a single input format. This project supports:

- Image files (.jpg, .png)
- Video files (.mp4, .avi)
- Live webcam feeds

This makes the system more flexible and widely applicable.

3. Visualization of Detection Results

For better interpretation of results, the system must display:

- Bounding boxes
- Class labels
- Confidence scores

This visual output helps users understand how well the system is performing.

4. Analytical Insights

Beyond detection, the system should provide statistical insights such as:

- Object frequency distribution
- Confidence score analysis
- Graphical representation of results

---

3. Objectives

The primary goal of this project is to design and implement a **real-time object detection system** using deep learning techniques.

The specific objectives include:

1. Implement YOLOv8-Based Detection

Develop a detection system using a pre-trained YOLOv8 model capable of identifying multiple objects within a single frame.

2. Support Multiple Input Sources

Enable detection from:

- Static images
- Pre-recorded videos

- Live webcam streams

This ensures versatility in different real-world scenarios.

3. Visualize Detection Outputs

Display results using:

- Rectangular bounding boxes
- Class labels
- Confidence scores

This helps verify the correctness of detections.

4. Analyze Detection Results

Generate graphical outputs such as:

- Bar charts showing object counts
- Histograms representing confidence scores

5. Develop an Efficient and User-Friendly System

Ensure the system is:

- Easy to use
- Modular in design
- Efficient in performance

---

4. Methodology

The methodology describes how the detection system works internally.

The core algorithm used is **YOLOv8**, which belongs to the family of **single-stage object detectors**.

Working Principle

1. Input Processing

The system accepts input from:

- Image files
- Video files
- Webcam streams

Images are resized to a fixed dimension (for example **640×640 pixels**) before being passed into the neural network.

---

2. YOLOv8 Model

The system uses the **pre-trained model**:

- **yolov8n.pt (Nano Model)**
- Trained on the **COCO dataset**

The COCO Dataset contains:

- 80 object categories
- Millions of labeled images
- Real-world scene diversity

Examples of detectable objects:

- Person
- Car
- Dog
- Bicycle
- Laptop

---

3. Single-Shot Detection

Unlike traditional two-stage detectors, YOLOv8 processes the entire image **in one pass**.

Advantages:

- Faster processing
- Reduced computation time
- Real-time performance

---

4. Object Detection Output

The model predicts:

- Object class label
- Confidence score
- Bounding box coordinates

Example Output:

Person — 0.92
Car — 0.87

---

5. Filtering

Low-confidence detections are removed using a threshold value such as:

Confidence Threshold = **0.5**

This improves detection quality.

---

6. Visualization

Detected objects are displayed using:

- Colored bounding boxes
- Text labels
- Confidence scores

---

7. Data Analysis

Detection results are stored and analyzed to produce:

- Object count distribution
- Confidence score trends

---

5. Workflow

The workflow explains the step-by-step execution of the system.

Step 1: Install Required Libraries

Required libraries include:

- Ultralytics
- OpenCV
- NumPy
- Matplotlib

Command:

pip install ultralytics opencv-python matplotlib numpy

Step 2: Load YOLOv8 Model

The YOLOv8 model is loaded into memory.

Example:

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
```

Step 3: Provide Input

Input can be:

- Image file
- Video file
- Webcam feed

Step 4: Process Input Data

Frames are resized and converted into required format.

Step 5: Run YOLOv8 Detection

The model performs detection on input data.

Step 6: Extract Object Details

Extract:

- Class name
- Confidence value
- Bounding box coordinates

Step 7: Draw Bounding Boxes

Boxes are drawn using OpenCV.

Step 8: Save Annotated Output

Detected outputs are stored in files.

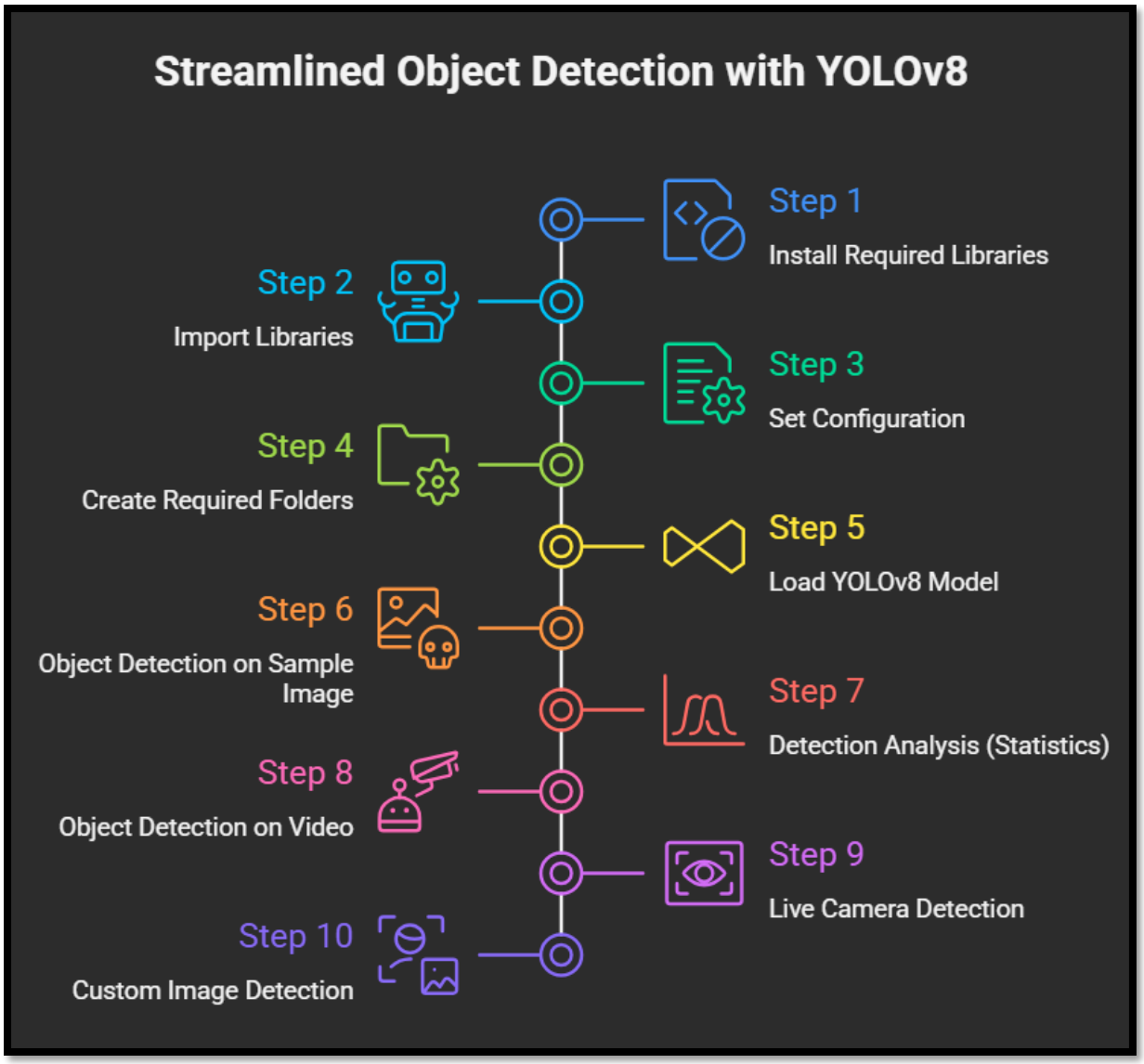
Step 9: Generate Graphical Analysis

Create:

- Bar charts
- Histograms

Step 10: Display Final Results

Outputs are displayed to the user.



---

6. System Design & Implementation

The system is implemented using **modular programming**, allowing each component to work independently.

Technologies Used (Expanded)

Python

Primary programming language used for:

- Model implementation

- Data processing
- Visualization

OpenCV

Used for:

- Image processing
- Video handling
- Drawing bounding boxes

Ultralytics YOLOv8

Provides:

- Pre-trained models
- Detection APIs

Matplotlib

Used for:

- Graph generation
- Visualization

NumPy

Used for:

- Matrix operations
- Numerical computations

Modules

1. Image Detection Module

Functions:

- Load image

- Perform detection
- Save annotated output

Example:

Input → image.jpg
Output → detected_image.jpg

---

2. Video Detection Module

Processes video frame-by-frame.

Steps:

- Read video frames
- Apply detection
- Combine frames into output video

---

3. Webcam Detection Module

Captures live feed from camera.

Features:

- Real-time detection
- Snapshot support

---

4. Analytics Module

Generates graphical outputs such as:

Bar Chart

Displays number of detected objects.

Example:

Person — 10
Car — 6

Histogram

Displays distribution of confidence scores.

---

7. Results and Discussion

The developed system was tested on various input formats including images, videos, and live webcam streams.

Performance Analysis

The system achieved:

- 20–30 Frames Per Second (FPS) on CPU
- Faster performance on GPU

Performance depends on:

- System hardware
- Input resolution
- Model size

Accuracy Analysis

The model showed high detection accuracy for:

- Large objects
- Clearly visible objects

Performance decreases when:

- Lighting is poor
- Objects are very small
- Objects overlap heavily

Outputs Generated

The system produces:

1. Annotated images
2. Processed videos
3. Statistical graphs

Limitations (Expanded)

1. Poor performance in low lighting
2. Difficulty detecting very small objects
3. Requires GPU for faster processing
4. Performance affected by motion blur

---

8. Advantages

- Real-time processing capability
- High detection accuracy
- Supports multiple input formats
- Easy to integrate into existing systems
- Lightweight models available
- Scalable architecture

---

9. Applications

Real-world applications include:

---

1. Smart Surveillance

Detect suspicious activities automatically.

Used in:

- Airports
- Railway stations
- Public places

---

2. Traffic Monitoring

Detect:

- Vehicles
- Pedestrians
- Traffic violations

---

3. Autonomous Vehicles

Used to detect:

- Road signs
- Vehicles
- Obstacles

---

4. Industrial Automation

Detect:

- Defective products
- Missing components

---

5. Security Systems

Used for:

- Face detection
- Intrusion detection

---

10. Conclusion

This project successfully demonstrates the implementation of a **real-time object detection system** using YOLOv8. The system achieves efficient performance by processing entire images in a single pass using deep learning techniques.

Compared to traditional detection methods, YOLOv8 provides:

- Faster processing speed
- Higher detection accuracy
- Better scalability

The modular design ensures easy expansion and integration with future technologies. Overall, this project highlights the effectiveness of modern deep learning models in solving complex visual recognition problems.

---

11. Future Scope

Possible future improvements include:

1. Custom Model Training

Train YOLOv8 on custom datasets.

Example:

- Medical images
- Industrial defects

---

2. Cloud Integration

Deploy system on cloud platforms for:

- Remote monitoring
- Data storage

---

3. Mobile Deployment

Implement detection on:

- Android devices
- Edge devices

---

4. Alert Systems

Trigger alerts when:

- Specific objects are detected
- Threshold values are exceeded

---

12. References

1. Ultralytics YOLOv8 Documentation
   https://docs.ultralytics.com/

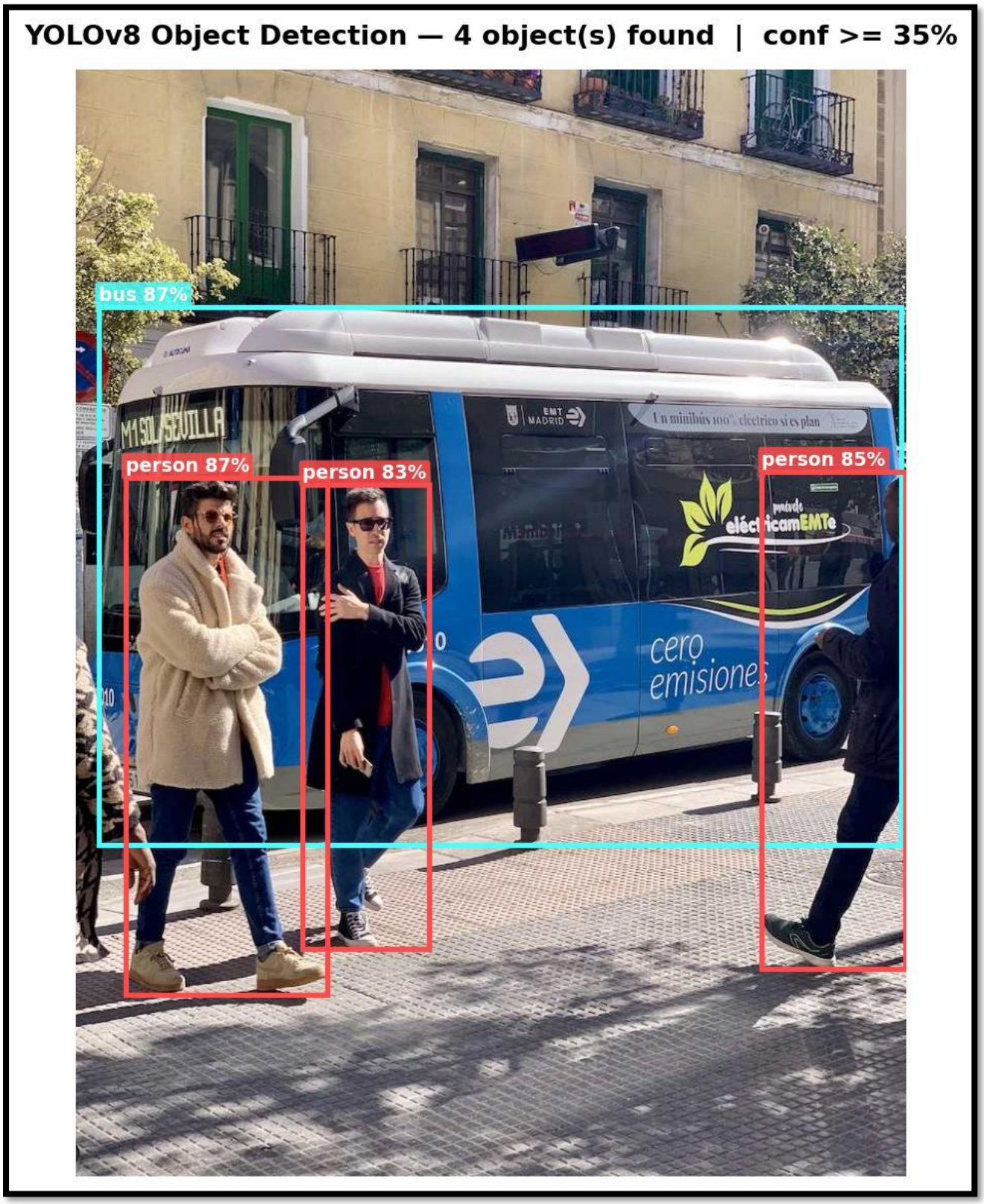
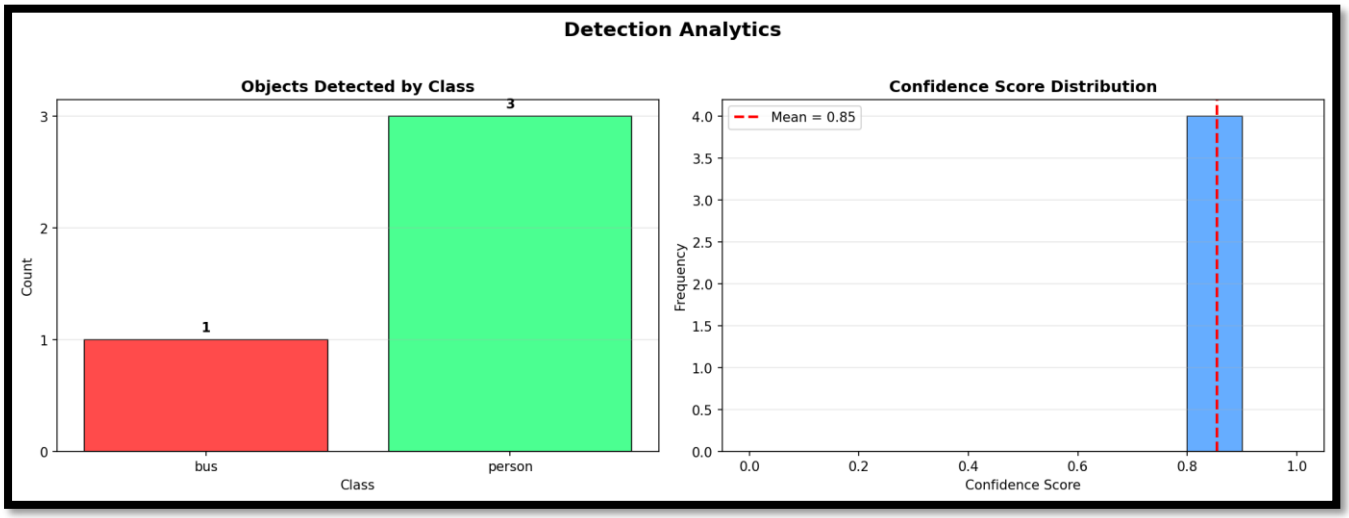
2. COCO Dataset
   https://cocodataset.org/
3. OpenCV Documentation
   https://docs.opencv.org/
4. Python Documentation
   https://www.python.org/
5. Redmon, J., et al. (2016).
   "You Only Look Once: Unified, Real-Time Object Detection"
   IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
6. Bochkovskiy, A., et al. (2020).
   "YOLOv4: Optimal Speed and Accuracy of Object Detection"

---

13. Output Samples

Webcam Detection Output

Image Detection Output



Detection Analysis